МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра МОЭВМ

ОТЧЕТ

по практической работе № 3
по дисциплине «Системы параллельной обработки данных»

Тема: «Поэлементное суммирование массивов средствами MPI»

Студент гр. 2304 _____ Пархоменко М. С.
подпись

Преподаватель _____ Татаринов Ю. С.
подпись

Санкт-Петербург
2026

1 Цель работы

Изучить коллективную операцию редукции MPI, реализовать поэлементное суммирование массивов целых чисел и исследовать время выполнения при изменении числа процессов в соответствии с постановкой задачи.

2 Формулировка задания

Вариант 7. В каждом процессе дан набор из $K + 5$ целых чисел, где K — количество процессов. Используя функцию MPI_Reduce для операции MPI_SUM, просуммировать элементы данных наборов с одним и тем же порядковым номером и вывести полученные суммы в главном процессе.

3 Описание алгоритма

В работе участвуют K процессов с рангами $r = 0, \dots, K - 1$. После MPI_Init каждый процесс узнаёт свой ранг и число процессов, создаёт массив длины $n = K + 5$ и заполняет его по правилу

$$a_{r,i} = ir, \qquad 0 \le i < n, \tag{1}$$

где $a_{r,i}$ — элемент с индексом i в процессе с рангом r.

После вывода исходных массивов выполняется MPI_Barrier. Затем все процессы вызывают MPI_Reduce с одинаковыми параметрами: число элементов n, тип MPI_INT, операция MPI_SUM, корневой процесс 0 и коммуникатор MPI_COMM_WORLD. При передаче массива редукция выполняется поэлементно, а приёмный буфер значим у корневого процесса [1].

Процесс 0 выводит массив сумм. Каждый процесс вызывает MPI_Finalize. Полный исходный код приведён в приложении А, а отдельная измерительная версия — в приложении Б.

Для проверки результата используется формула

$$b_i = \sum_{r=0}^{K-1} a_{r,i} = i\frac{K(K-1)}{2}, \qquad 0 \le i < K + 5, \tag{2}$$

где b_i — элемент результирующего массива с индексом i. При $K = 1$ исходный массив нулевой, поэтому результат также нулевой. Подготовка данных занимает $O(n)$ операций на процесс; каждый процесс хранит два массива длины n. Внутренний способ выполнения редукции выбирается библиотекой MPI и в исходном коде программы не задаётся [1].

4 Формальное описание посредством сети Петри

Алгоритм представлен цветной сетью Петри на рисунке 1. Фишка содержит ранг процесса и, при необходимости, его массив. Начальная маркировка места I содержит K фишек с различными рангами; она соответствует состоянию после инициализации MPI. Для описания параллельных процессов использован аппарат сетей Петри [2].



Рисунок 1 – Цветная сеть Петри основной программы

Переход t_p срабатывает отдельно для каждого ранга, остальные переходы отражают общие этапы или действие корневого процесса. Функционал переходов приведён в таблице 1. Число у дуги означает количество фишек; немаркированная дуга переносит одну фишку. Ветвь с весом $K - 1$ при $K = 1$ не создаёт фишек.

Таблица 1 – Функционал переходов сети Петри

Переход	Действие
t_p	Извлекает фишку ранга r из I, формирует и выводит массив a_r, помещает фишку (r, a_r) в A.
t_b	При наличии всех K фишек из A переносит их в B; моделирует барьер перед коллективной операцией.
t_r	Обрабатывает K массивов из B, вычисляет $b_i = \sum_r a_{r,i}$; создаёт фишку $(0, b)$ в R и $K - 1$ фишек ненулевых рангов в D.
t_o	Выводит массив из фишки $(0, b)$ места R и переносит фишку ранга 0 в D.
t_f	При наличии K фишек в D переводит модель в конечное состояние F; обозначает завершение программы всеми процессами.

Редукция показана одним укрупнённым переходом, без моделирования внутренних сообщений библиотеки. Модель отражает результат операции и не задаёт одновременный выход всех процессов из MPI_Reduce. Место D и переход t_f фиксируют завершение всей программы; дополнительного вызова барьера после редукции в коде нет.

Каждый ранг проходит подготовку один раз и участвует в коллективной операции. Из R выходит только фишка ранга 0, поэтому вывод массива сумм выполняется один раз. После обработки корня в D находятся все K рангов, что позволяет достичь конечного состояния.

5 Примеры работы и проверка корректности

На рисунке 2 показан запуск на четырёх процессах. В каждом процессе сформирован массив из девяти чисел. Сумма рангов равна 0 + 1 + 2 + 3 = 6, поэтому по формуле (2) ожидается массив

$$(0, 6, 12, 18, 24, 30, 36, 42, 48).$$

На скриншоте получен именно этот результат.

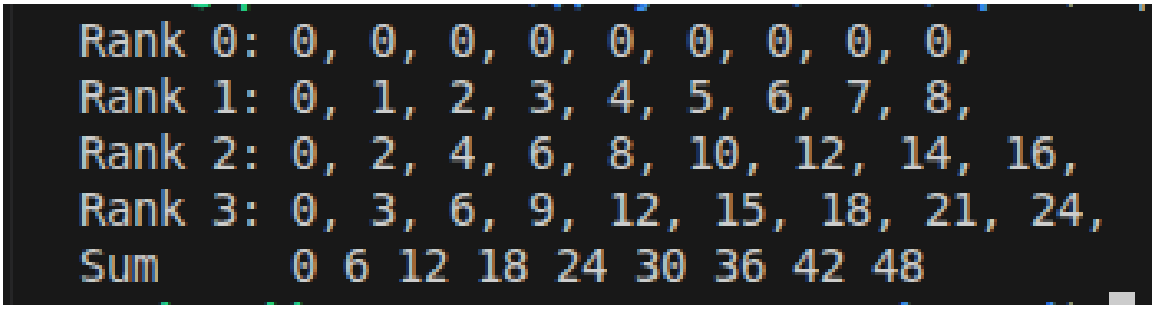


Рисунок 2 – Исходные массивы и результат на четырёх процессах

Результаты функциональных проверок представлены в таблице 2. В этих запусках результаты соответствовали формуле (2), программы завершились с кодом 0.

Таблица 2 – Функциональные проверки программы

K	n	Полученный массив сумм
1	6	$(0, 0, 0, 0, 0, 0)$
2	7	$(0, 1, 2, 3, 4, 5, 6)$
4	9	$(0, 6, 12, 18, 24, 30, 36, 42, 48)$

Порядок строк разных рангов может изменяться. Барьер перед редукцией ожидает участия всех процессов, но не упорядочивает доставку их вывода в терминал. Эта особенность не влияет на вычисленные суммы.

6 Экспериментальное исследование

6.1. Методика и условия

Измерительная версия сохраняет длину массива $n = K + 5$. Подготовка массивов, барьер, инициализация и завершение MPI, а также печать не включаются в измеряемый интервал. Каждый процесс измеряет локальное время серии из $L = 1000$ вызовов MPI_Reduce с операцией MPI_SUM. Таймер MPI_Wtime вызывается непосредственно перед циклом и сразу после него.

Отдельный коллективный вызов с MPI_MAX собирает максимум локальных длительностей после остановки таймера. Время одного вызова в запуске j вычисляется по формуле

$$T_j(K) = \frac{\max_{0 \le r < K} (t_{r,j}^{\text{end}} - t_{r,j}^{\text{start}})}{L}. \tag{3}$$

Таким образом, часы разных процессов друг с другом не вычитаются.

Для каждого $K = 1, 2, 4, 8, 16$ выполнено семь отдельных запусков MPI; итоговым временем считается медиана семи значений $T_j(K)$. Прогрев не выполнялся: начальные затраты первых вызовов входят в длительность серии. В каждой серии повторно обрабатываются одни и те же буферы; результат перезаписывается и не накапливается между итерациями.

Скрипт pr3/scripts/run_benchmark.py сохраняет все 35 значений в CSV, составляет сводную таблицу и записывает параметры окружения. Пример прогона приведён на рисунке 3.

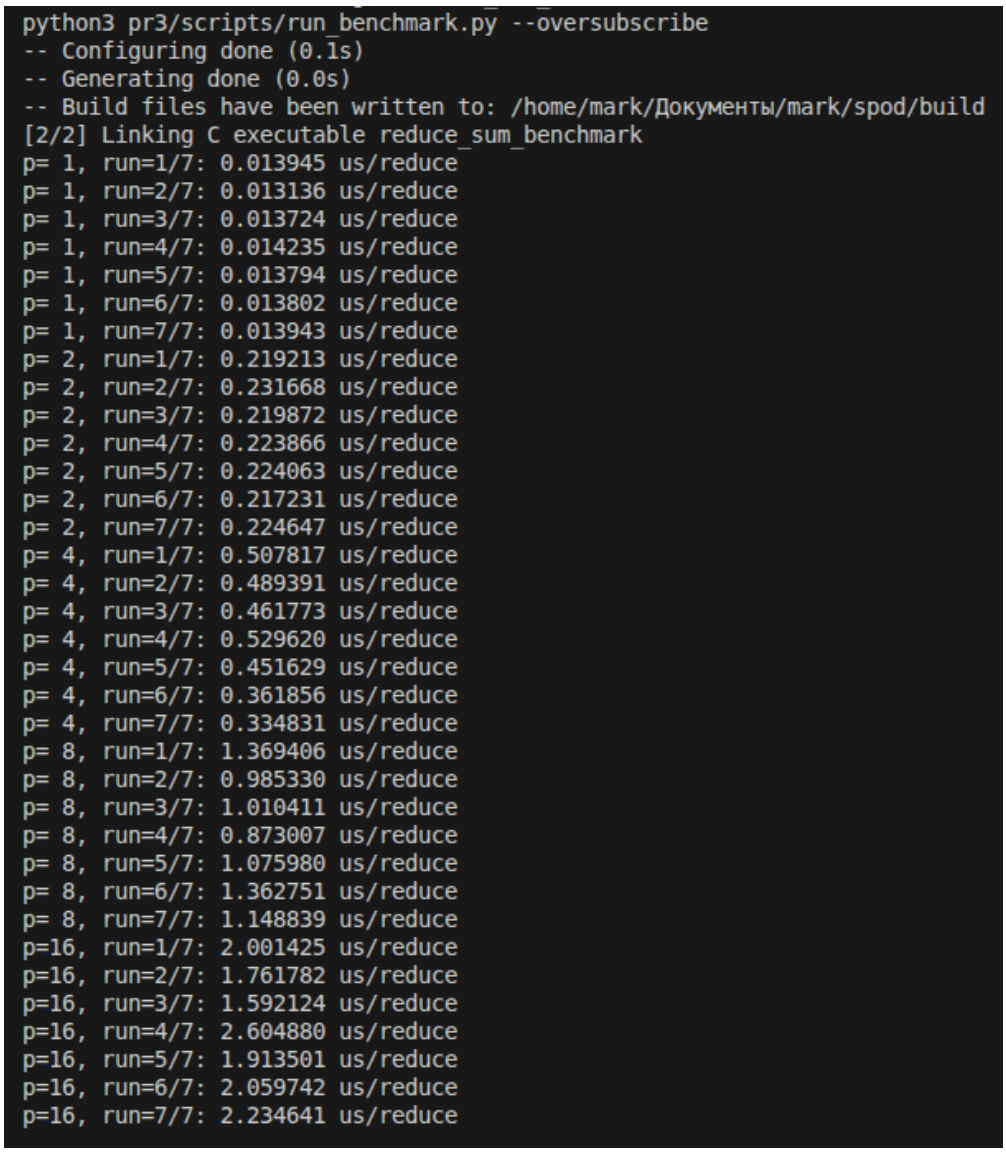


Рисунок 3 – Семь запусков бенчмарка для каждого количества процессов

Согласно сохранённым метаданным использованы Intel Core i3-12100, восемь логических CPU, Linux 7.0.0-42-generic, Open MPI 4.1.6 и GCC 13.3.0. Конфигурация CMake — Release, параметры оптимизации -O3 -DNDEBUG. Запуск выполнялся с параметрами --use-hwthread-cpus --oversubscribe. При $K = 16$ число процессов превышало число логических CPU, то есть использовалась переподписка.

6.2. Результаты измерений

В таблице 3 приведены времена одного вызова в микросекундах. Минимум и максимум характеризуют разброс между семью запусками, а не

времена отдельных итераций внутри серии.

Таблица 3 – Время одной редукции: семь запусков без прогрева, мкс

K	$n = K + 5$	Минимум	Медиана	Максимум
1	6	0,013	0,014	0,014
2	7	0,217	0,224	0,232
4	9	0,335	0,462	0,530
8	13	0,873	1,076	1,369
16	21	1,592	2,001	2,605

Зависимость времени от числа процессов показана на рисунке 4. Вертикальные отрезки обозначают минимум и максимум, точки — медиану. Логарифмическая шкала времени позволяет показать режим без обмена при $K = 1$ и режимы с несколькими процессами.

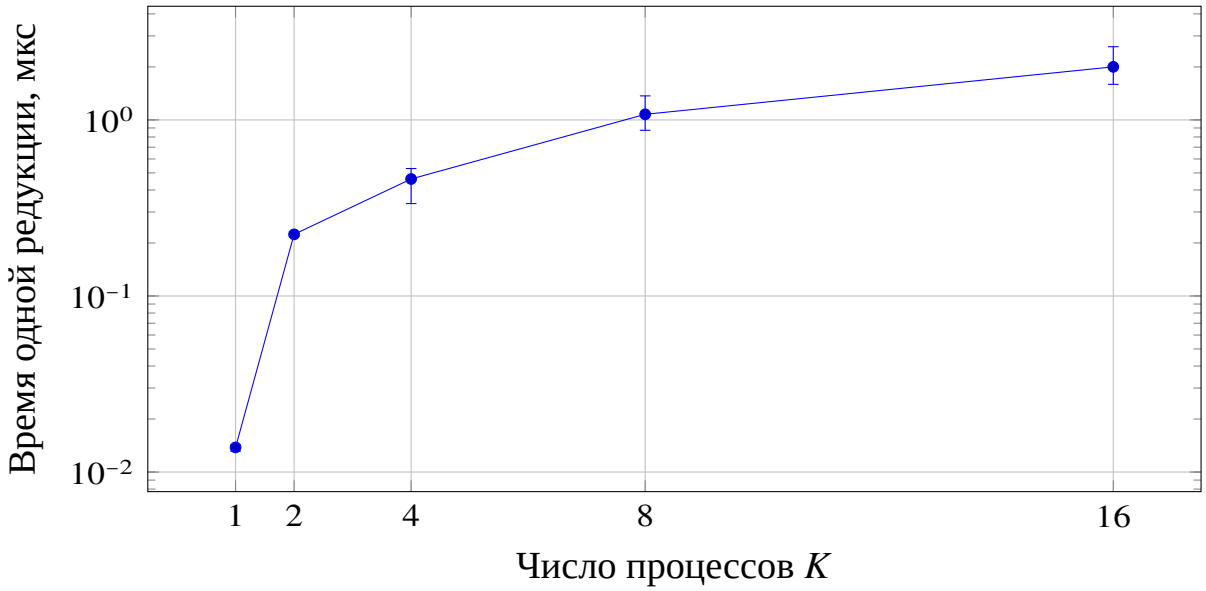


Рисунок 4 – Время редукции при изменении K и длине массива $n = K + 5$

На рисунке 5 те же измерения представлены по длине локального массива. Это не независимая серия по размеру задачи: каждой длине соответствует своё число процессов.

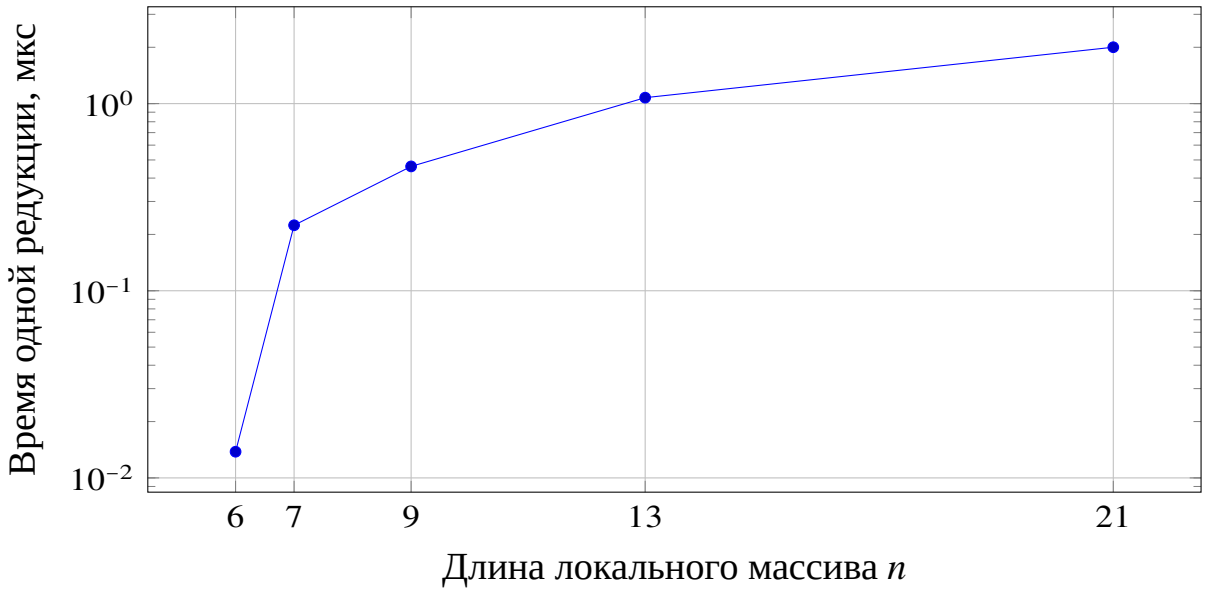


Рисунок 5 – Время для массивов длины $n = K + 5$ при соответствующих K

6.3. Анализ времени и границы исследования

В рассмотренных точках медианное время монотонно увеличивалось: от 0,013802 мкс при $K = 1$ до 2,001425 мкс при $K = 16$. При $K = 1$ межпроцессной передачи нет; этот режим значительно дешевле вариантов с несколькими процессами. При переходе от $K = 2$ к $K = 4$ медиана выросла примерно в 2,06 раза, от $K = 4$ к $K = 8$ — в 2,33 раза, от $K = 8$ к $K = 16$ — в 1,86 раза.

При $K = 4$ времена отдельных запусков лежали в диапазоне 0,334831–0,529620 мкс, при $K = 8$ — 0,873007–1,369406 мкс. Разброс показывает, что результат зависит от условий конкретного запуска. На него могут влиять планирование процессов, начальные затраты и состояние кешей. Увеличение числа участников добавляет расходы коллективного обмена; при этом длина локального массива также возрастает с 6 до 21 элемента. Отдельный вклад каждого фактора по этим данным определить нельзя.

Точка $K = 16$ дополнительно включает переподписку. Её нельзя трактовать как выполнение на шестнадцати независимых аппаратных потоках. По изменению наклона графика нельзя определить, менялся ли

внутренний алгоритм MPI. Для этого нужны отдельные опыты.

Общие требования предусматривают независимое исследование по числу процессов и размеру задачи. В текущей программе $n = K + 5$, поэтому получена только одна связанная серия. Для полного исследования нужны отдельно задаваемые значения n, например 16, 1024 и 65536, с одинаковым набором K для каждого n. Эти эксперименты пока не проведены.

6.4. Ускорение и эффективность

Для сопоставимых объёмов работы ускорение и эффективность определяются как

$$S(K,n) = \frac{T_{\text{seq}}(K,n)}{T_{\text{MPI}}(K,n)}, \qquad E(K,n) = \frac{S(K,n)}{K}, \tag{4}$$

где $T_{\text{seq}}(K,n)$ — время последовательного суммирования тех же K массивов длины n, а $T_{\text{MPI}}(K,n)$ — время MPI-варианта без формирования исходных данных и вывода.

Редукция на одном процессе обрабатывает только один массив и не является такой последовательной базой. Поэтому отношение измеренного времени при $K = 1$ к времени при $K > 1$ не характеризует ускорение одной задачи. Последовательная база не измерялась: численные значения и графики S и E по имеющимся данным не строятся. Зону эффективного распараллеливания на основании этой серии установить нельзя.

7 Выводы

Реализовано поэлементное суммирование массивов длины $K + 5$ с помощью MPI_Reduce и MPI_SUM. Все процессы участвуют в коллективном вызове, а массив сумм получает и выводит процесс 0. Проверки при $K = 1, 2, 4$ дали результаты, соответствующие аналитической формуле. Алгоритм представлен цветной сетью Петри с описанием функционала переходов.

Время исследовано для $K = 1, 2, 4, 8, 16$: выполнено по семь запусков, в каждом измерена серия из 1000 редукций без прогрева. Измеряемый участок не включает подготовку массивов, вывод и сбор локальных длительностей. Медиана времени одного вызова возросла с 0,013802 до 2,001425 мкс. Увеличение числа процессов в этой серии не сопровождалось уменьшением времени; при 16 процессах использовалась переподписка.

Полученные данные характеризуют текущую постановку со связанными K и n, но не дают независимых зависимостей по двум параметрам. Для завершения исследования эффективности требуются серии с независимо заданной длиной массива и измерения сопоставимой последовательной базы. До их выполнения вывод о наличии или отсутствии области ускорения не обоснован.

8 Список использованных источников

[1] MPI Forum. MPI 4.1. Раздел 7.9.1. Reduce. 2023. https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report/node130.htm.
Дата обращения: 10.11.2026.

[2] Мараховский В. Б., Розенблюм Л. Я., Яковлев А. В. Моделирование параллельных процессов. Сети Петри. СПб.: Профессиональная литература, 2014.

ПРИЛОЖЕНИЕ А

Листинг основной программы

```
// Автор: Марк Пархоменко
// Дата: 10.11.2026

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#include <mpi.h>

#define MAGIC_NUM 5

int main(int argc, char *argv[])
{
    int rank;
    int process_count;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &process_count);

    const size_t count_nums = process_count + MAGIC_NUM;
    int nums_send[count_nums];

    printf("Rank %d: ", rank);
    for (size_t i = 0; i < count_nums; i++)
```

```
{
    nums_send[i] = i * rank;
    printf("%d, ", nums_send[i]);
}
printf("\n");

MPI_Barrier(MPI_COMM_WORLD);

int nums_recv[count_nums];

MPI_Reduce(
    &nums_send,
    &nums_recv,
    count_nums,
    MPI_INT,
    MPI_SUM,
    0,
    MPI_COMM_WORLD
);

if (rank == 0)
{
    printf("Sum     ");
    for (size_t i = 0; i < count_nums; i++)
    {
        printf("%d ",
            nums_recv[i]);
    }
```

```
        printf("\n");
    }

    MPI_Finalize();
    return 0;
}
```

ПРИЛОЖЕНИЕ Б

Листинг измерительной программы

```
// Автор: Марк Пархоменко
// Дата: 10.11.2026

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#include <mpi.h>

#define MAGIC_NUM 5
#define COUNT_ITER 1000

int main(int argc, char *argv[])
{
    int rank;
    int process_count;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &process_count);

    const size_t count_nums = process_count + MAGIC_NUM;
    int nums_send[count_nums];

    for (size_t i = 0; i < count_nums; i++)
```

```
{
    nums_send[i] = i * rank;
}
int nums_recv[count_nums];

MPI_Barrier(MPI_COMM_WORLD);

// Начало замеров
double start_time = MPI_Wtime();

for (int iter = 0; iter < COUNT_ITER; iter++) // COUNT_ITER итераций
  ↪ вызова MPI_Reduce
{
    MPI_Reduce(
        &nums_send,
        &nums_recv,
        count_nums,
        MPI_INT,
        MPI_SUM,
        0,
        MPI_COMM_WORLD
    );
}

double elapsed_time = MPI_Wtime() - start_time;

// Конец замеров
```

```
    double max_elapsed;
    MPI_Reduce(
        &elapsed_time,
        &max_elapsed,
        1,
        MPI_DOUBLE,
        MPI_MAX,
        0,
        MPI_COMM_WORLD
    );

    if (rank == 0)
    {
        double avg_max_time = max_elapsed / COUNT_ITER;
        printf("Avg max time %.12f seconds\n",
            avg_max_time);
    }

    MPI_Finalize();
    return 0;
}
```